

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



VI XỬ LÝ-VI ĐIỀU KHIỂN (CO3009)

BÀI TẬP LỚN

TRAFFIC LIGHT CONTROLLER

GVHD: CAO TIẾN ĐẠT

Sinh viên thực hiện: Trần Nhật Minh - 2212085 (Lớp L03)
Nguyễn Nhật Nam - 2212147 (Lớp L03)
Vũ Đức Lâm - 2211824 (Lớp L03)

Link GitHub: https://github.com/Raven1911/Assignment_VXL_VDK/tree/main

Theo dõi video để quan sát quá trình kiểm thử: https://www.youtube.com/watch?v=-_3_lybrcdA

THÀNH PHỐ HỒ CHÍ MINH 2024



Mục lục

Figures	3
Bảng	3
DANH SÁCH THÀNH VIÊN	3
1 Giới thiệu	4
2 Ý tưởng	5
2.1 Danh sách thiết bị	5
2.2 Nút nhấn	5
2.3 Finite State Machine	5
2.4 Cấu hình GPIO, giao tiếp I2C và Timer	8
3 Thiết kế và hiện thực	11
3.1 Bộ định thời	11
3.2 Danh sách hàm bộ định thời	11
3.3 Giao tiếp I2C	12
4 Kiểm thử	14



Figures

1	FSM của hệ thống	6
2	Cấu hình các chân GPIO	8
3	Giao tiếp I2C	9
4	Timer của thiết bị	10
5	Cấu trúc dữ liệu cho bộ định thời	11
6	Hàm trong file <code>i2c-lcd.c</code>	12
7	Hàm trong file <code>i2c-lcd.c</code>	13
8	Nội dung hàm main	14

Bảng

1	DANH SÁCH THÀNH VIÊN VÀ PHÂN CHIA CÔNG VIỆC	3
2	Danh sách trang thiết bị sử dụng	5
3	Danh sách các hàm bộ định thời	12



DANH SÁCH THÀNH VIÊN VÀ PHÂN CHIA CÔNG VIỆC

No.	Họ và Tên	MSSV	Nhiệm vụ	Mức độ hoàn thành
1	Trần Nhật Minh	2212085	- Ý tưởng - Thiết kế	100%
2	Nguyễn Nhật Nam	2212147	- Hỗ trợ - Report	100%
3	Vũ Đức Lâm	2211824	- Thiết kế - Kiểm thử	100%

Table 1: DANH SÁCH THÀNH VIÊN VÀ PHÂN CHIA CÔNG VIỆC

1 Giới thiệu

Trong kỷ nguyên đồ án toàn giao thông hiện nay, hệ thống đèn giao thông đóng vai trò quan trọng trong việc duy trì trật tự và hạn chế tình trạng ùn tắc. Trong số nhiều phương án công nghệ hiện đại, việc sử dụng vi điều khiển STM32 để lập trình hệ thống đèn giao thông hai chiều đang trở thành một giải pháp được ưa chuộng nhờ đặc tính linh hoạt, độ chính xác cao, và chi phí hiệu quả.

STM32 là dòng vi điều khiển ARM Cortex-M do STMicroelectronics sản xuất, được biết đến nhờ hiệu năng xử lý mạnh mẽ, tính đa dạng và khả năng tích hợp cao. Nhờ vào sự độc đáo của STM32 trong việc quản lý GPIO (General Purpose Input/Output) và đồng bộ hóa thông qua Timer, bộ điều khiển đèn giao thông hai chiều trở nên đơn giản và dễ triển khai.

Trong bài toán điều khiển đèn giao thông hai chiều, STM32 được lập trình để quản lý hầu hết các khía cạnh quan trọng như:

1. Thời gian chuyển đổi đèn: Dựa trên Timer tích hợp, STM32 có thể định thời gian chính xác cho mỗi giai đoạn đèn xanh, vàng, đỏ.
2. Kiểm soát tín hiệu LED: STM32 quản lý các LED giao thông thông qua các chân GPIO, cho phép bật/tắt LED tại các thời điểm đã xác định.
3. Tính đồng bộ hóa hai chiều: STM32 cho phép quản lý đồng bộ thời gian và luân chuyển tín hiệu giữa hai hướng giao thông một cách linh hoạt.

Hơn nữa, STM32 được hỗ trợ bởi các công cụ như **STM32CubeIDE** và **STM32CubeMX**, giúp đơn giản hóa việc cấu hình phần cứng và lập trình firmware. Tính năng được nhóm sử dụng chính là giao thức I2C để giao tiếp với các module ngoại vi như các cảm biến giao thông hoặc module LCD cũng đem lại tính năng mở rộng cho bài toán.

Tóm lại, việc sử dụng STM32 trong lập trình bộ điều khiển đèn giao thông hai chiều là một điểm nhấn quan trọng trong việc áp dụng công nghệ để giải quyết bài toán giao thông hiện đại. Sự kết hợp giữa tính năng vượt trội của STM32 và khả năng tích hợp các yếu tố bên ngoài hứa hẹn mang lại một hệ thống đèn giao thông hiệu quả và đồng bộ.

2 Ý tưởng

Trong bài tập lớn này, nhóm sẽ hiện thực đèn giao thông 2 chiều với các chức năng sau:

1. Chế độ tự động (Automatic mode)
2. Chế độ điều chỉnh (Manual mode)

Ngoài ra nhằm đáp ứng yêu cầu bài tập lớn, nhóm sử dụng giao tiếp I2C (Inter-Integrated Circuit) thông qua LCD khi mô phỏng trên STM32F103RB.

2.1 Danh sách thiết bị

Table 2: Danh sách trang thiết bị sử dụng

Thiết bị	Số lượng	Mục đích sử dụng
Đèn LED đơn xanh lá	2	Đèn phương tiện (2 phía)
Đèn LED đơn màu đỏ	2	Đèn phương tiện (2 phía)
Đèn LED đơn màu vàng	2	Đèn phương tiện (2 phía)
Nút nhấn	3	Điều khiển qua lại giữa các chế độ đèn của phương tiện
Board NUCLEO STM32F103RB	1	Điều khiển hệ thống đèn giao thông
LCD 1602 + IC I2C	1	Hiển thị trạng thái đèn

2.2 Nút nhấn

Gồm các nút nhấn để thay đổi giữa các chế độ được nêu

Hiện thực cơ chế nhấn đèn để: tự động tăng/giảm giá trị khi ở chế độ điều chỉnh thời lượng sáng các đèn, hoặc tự động nhảy sang trạng thái con/chế độ tiếp theo.

2.3 Finite State Machine

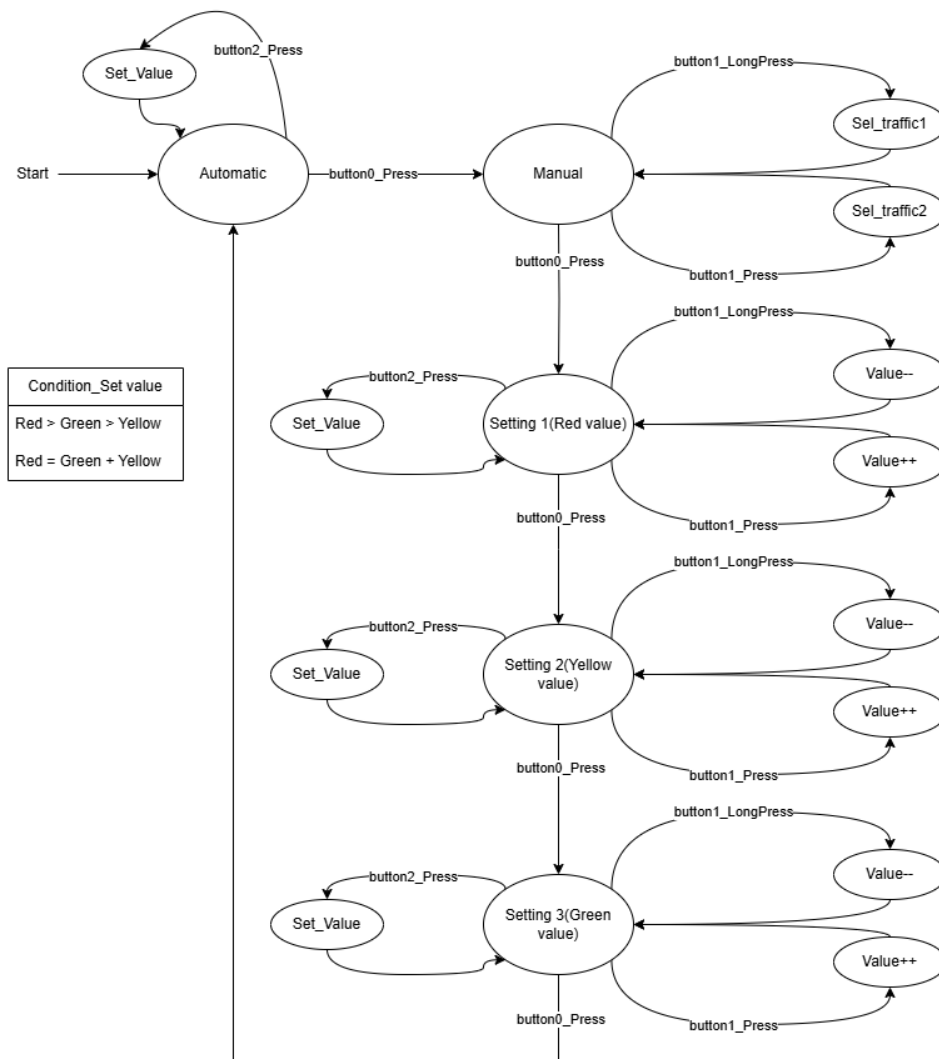


Figure 1: FSM của hệ thống



Giải thích: Ban đầu, đèn sẽ chạy tự động, nút nhấn là tín hiệu chuyển sang chế độ thủ công, điều chỉnh và trở lại tự động. Cơ chế hoạt động của từng chế độ sẽ như sau:

1. **Tự động:** Đèn của mỗi phía sẽ được khởi tạo với màu đối lập nhau (một bên xanh, một bên đỏ) và bắt đầu chạy theo vòng lặp: xanh $\rightarrow$ vàng $\rightarrow$ đỏ, rồi quay ngược về xanh với thời gian sáng mỗi màu theo như thời gian đã khởi tạo (có thể thay đổi trong cơ chế điều chỉnh).
2. **Thủ công:** Ở chế độ này, màu đèn phương tiện của mỗi phía sẽ hoàn toàn do người điều khiển giao thông điều chỉnh thủ công, vì vậy, sẽ phải **cực kỳ cẩn trọng**. Khi chuyển sang chế độ này, hai phía sẽ được khởi tạo với màu đối lập nhau (một bên đỏ, một bên xanh). Mỗi lượt nhấn nút, màu của đèn sẽ chuyển từ: xanh $\rightarrow$ vàng $\rightarrow$ đỏ, và quay ngược lại ban đầu.
3. **Điều chỉnh:** Mỗi màu đèn sẽ có thời gian sáng khác nhau, chế độ này được dùng để điều chỉnh thời gian. Một nút sẽ để chọn màu muốn chỉnh thời gian, hai nút dùng để tăng/giảm.

2.4 Cấu hình GPIO, giao tiếp I2C và Timer

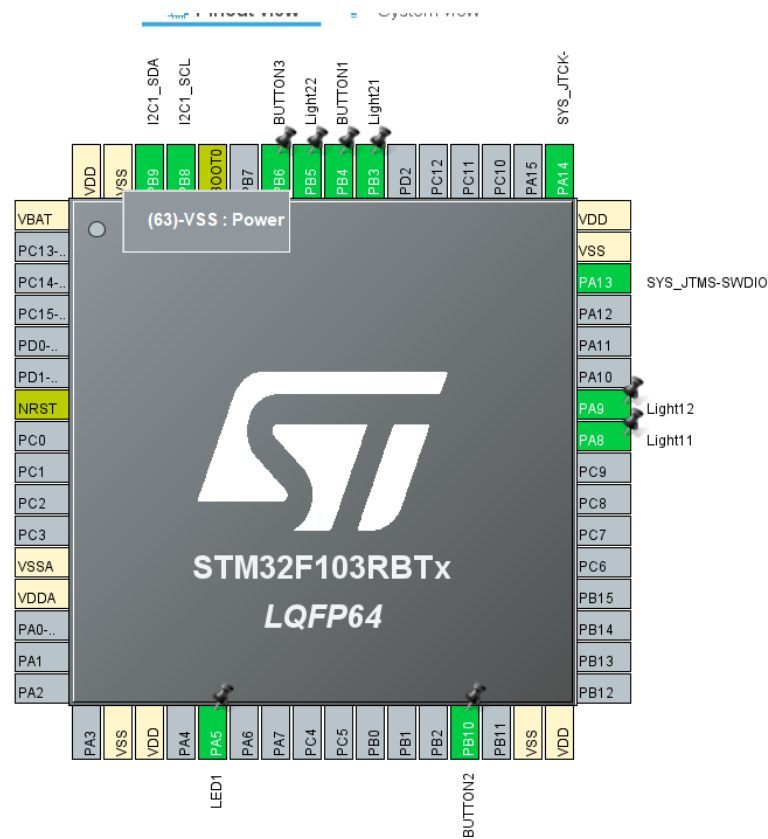


Figure 2: Cấu hình các chân GPIO

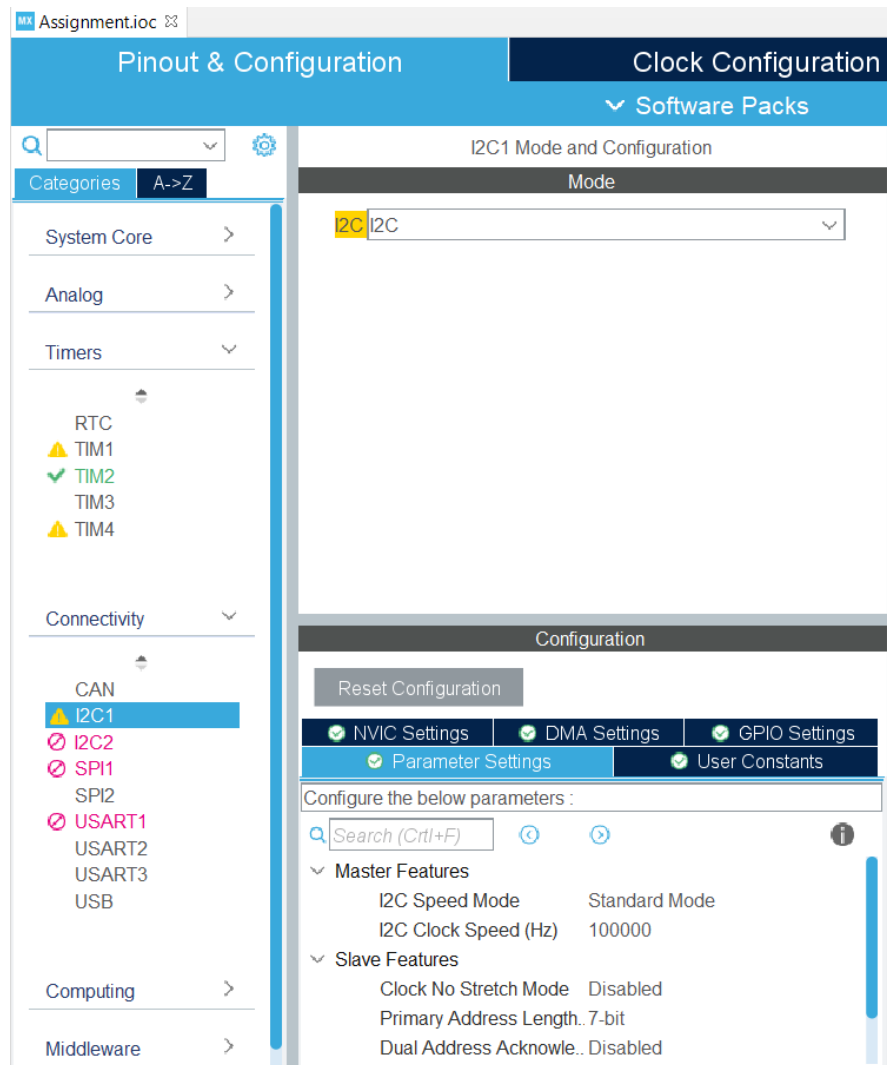


Figure 3: Giao tiếp I2C

Giao tiếp I2C của hệ thống được thiết kế theo chế độ tiêu chuẩn (Standard Mode) với Clock Speed là 100000 Hz.

Hệ thống được sử dụng TIMER2 để cấu hình như sau:

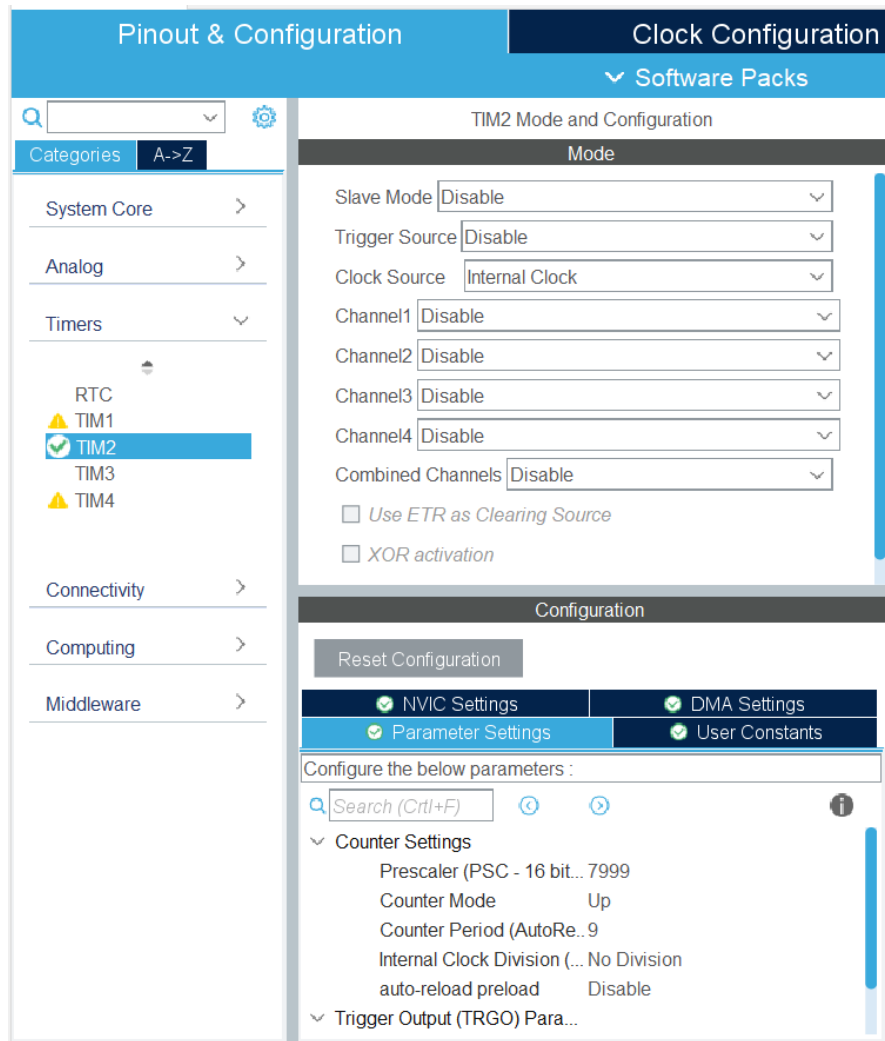


Figure 4: Timer của thiết bị

Hệ thống được cài đặt Prescaler và Counter Period lần lượt là 7999 và 9. Đồng thời tại NVIC Settings, Timer2 đã được enable.

3 Thiết kế và hiện thực

3.1 Bộ định thời

Cấu trúc dữ liệu

```
18 #define NUM_TASKS 20
19
20 typedef struct TaskNode {
21     void (* pTask)(void);
22     uint8_t taskID;
23     uint32_t delay;
24     uint32_t period;
25     struct TaskNode* next;
26 } TaskNode;
27
28
29
30 typedef struct ListTasks {
31     TaskNode* tail;
32     uint32_t numTask;
33     uint32_t total_delay;
34     uint32_t emptyslot;
35 } ListTasks;
36
37
38
39 TaskNode* create_new_node(void (*pfunc)(), uint32_t delay, uint32_t period);
40
41 void sche_init();
42 void sche_update();
43 void sche_dispatcher();
44 uint8_t sche_add_task(void (*pfunc)(), uint32_t delay, uint32_t period);
45 void sche_delete_task(uint8_t taskID);
```

Figure 5: Cấu trúc dữ liệu cho bộ định thời

Đặc tả cấu trúc dữ liệu của một tác vụ được lưu trữ

- **pTask:** Con trỏ hàm của tác vụ (bắt buộc có kiểu hàm `void(void)`). Nếu `functionPointer == 0` (trỏ đến null), tác vụ chưa được thiết lập và ngược lại.
- **taskID:** Mã số định danh của mỗi tác vụ. Trong suốt thời gian tồn tại, mỗi tác vụ có một mã số định danh duy nhất.
- **delay:** Khoảng thời gian (ms) cho đến lần thực thi kế tiếp của tác vụ.
- **period:** Khoảng thời gian (ms) giữa hai lần chạy liên tiếp của tác vụ. Nếu `period == 0`, tác vụ chỉ được thực hiện một lần duy nhất.

3.2 Danh sách hàm bộ định thời

Tên hàm	Tham số	Ý nghĩa	Độ phức tạp
sche_init	void	Khởi tạo giá trị cho mảng, với thông số id được gán giá trị cụ thể, đảm bảo tính duy nhất để thuận tiện cho các giải thuật thêm/xóa.	$O(1)$
sche_update	void	Kiểm tra phần tử đầu tiên mỗi lần ngắt timer và cập nhật giá trị delay và flag (nếu tác vụ đã được khởi tạo).	$O(1)$
sche_dispatcher	void	Kiểm tra phần tử đầu tiên nếu sẵn sàng sẽ thực thi, xóa khỏi mảng và bổ sung lại tác vụ vào mảng (nếu là task lặp lại).	$O(1)$
sche_add_Task	void (*pfunc)(void), uint32_t delay, uint32_t period	Kiểm tra dung lượng mảng lưu trữ, tìm kiếm vị trí thích hợp để thêm tác vụ vào mảng bằng cách duyệt từng phần tử để dời về sau và chèn tác vụ vào vị trí mong muốn.	$O(n)$
sche_delete_task	uint8_t taskID	Duyệt từng phần tử để tìm kiếm và xóa tác vụ với id tương ứng. Giá trị id được tái sử dụng cho lần sau.	$O(n)$

Table 3: Danh sách các hàm bộ định thời

3.3 Giao tiếp I2C

```
void lcd_send_data (char data)
{
    char data_u, data_l;
    uint8_t data_t[4];
    data_u = (data >> 4) & 0xf0;
    data_l = (data << 4) & 0xf0;
    data_t[0] = data_u | 0x00; // en=1, rs=0
    data_t[1] = data_u | 0x09; // en=0, rs=0
    data_t[2] = data_l | 0x00; // en=1, rs=0
    data_t[3] = data_l | 0x09; // en=0, rs=0
    HAL_I2C_Master_Transmit (&hi2c1, SLAVE_ADDRESS_LCD, (uint8_t *) data_t, 4, 100);
}

void lcd_init (void) {
    lcd_send_cmd (0x33); /* set 4-bits interface */
    lcd_send_cmd (0x32);
    HAL_Delay(50);
    lcd_send_cmd (0x28); /* start to set LCD function */
    HAL_Delay(50);
    lcd_send_cmd (0x01); /* clear display */
    HAL_Delay(50);
    lcd_send_cmd (0x06); /* set entry mode */
    HAL_Delay(50);
    lcd_send_cmd (0x0c); /* set display to on */
    HAL_Delay(50);
    lcd_send_cmd (0x02); /* move cursor to home and set data address to 0 */
    HAL_Delay(50);
    lcd_send_cmd (0x80);
}
```

Figure 6: Hàm trong file i2c-lcd.c

```
void lcd_send_string (char *str)
{
    while (*str) lcd_send_data (*str++);
}

void lcd_clear_display (void)
{
    lcd_send_cmd (0x01); //clear display
    // HAL_Delay(50);
}

void lcd_goto_XY (int row, int col)
{
    uint8_t pos_Addr;
    if(row == 1)
    {
        pos_Addr = 0x80 + row - 1 + col;
    }
    else
    {
        pos_Addr = 0x80 | (0x40 + col);
    }
    lcd_send_cmd(pos_Addr);
}
```

Figure 7: Hàm trong file i2c-lcd.c

Tóm tắt chức năng tổng quát của hàm trong i2c-lcd.c

File i2c-lcd.c cung cấp tập hàm để giao tiếp và điều khiển màn hình LCD thông qua giao thức I2C. Nó tích hợp nhiều chức năng để hiển thị nội dung văn bản một cách hiệu quả trong các hệ thống nhúng, chẳng hạn như STM32. Dưới đây là tóm tắt chức năng của các hàm trong file:

- **Khởi tạo LCD (lcd_init):** Hàm này thiết lập màn hình LCD để sử dụng giao thức 4-bit qua I2C. Nó gửi các lệnh cơ bản như xác định chế độ giao tiếp, xóa màn hình, thiết lập chế độ hiển thị (2 dòng, font 5x8), đặt con trỏ về đầu dòng. Hàm đảm bảo màn hình LCD sẵn sàng hoạt động.
- **Gửi lệnh (lcd_send_cmd):** Hàm gửi các lệnh từ vi điều khiển đến màn hình LCD thông qua giao thức I2C. Các lệnh bao gồm xóa màn hình, di chuyển con trỏ, hay thay đổi chế độ hiển thị. Điều này giúp điều khiển trực tiếp các tính năng của màn hình.
- **Gửi dữ liệu (lcd_send_data):** Hàm này chuyển dữ liệu (các ký tự ASCII) đến LCD để hiển thị. Khi nhận dữ liệu này, màn hình hiển thị tự động tại vị trí con trỏ hiện tại, giúp hiển thị văn bản dễ dàng.
- **Gửi chuỗi (lcd_send_string):** Hàm gửi một chuỗi văn bản liên tục đến màn hình LCD. Nó sử dụng hàm lcd_send_data cho từng ký tự trong chuỗi, giúp nhanh chóng hiển thị thông tin.
- **Xóa màn hình (lcd_clear_display):** Gửi lệnh xóa toàn bộ màn hình LCD, lấy lại trạng thái ban đầu. Hàm này hữu ích khi cần xóa bỏ nội dung cũ để hiển thị nội dung mới.
- **Đặt vị trí con trỏ (lcd_goto_XY):** Hàm cho phép di chuyển con trỏ đến vị trí mong muốn (dòng và cột). Các vị trí được tính dựa trên địa chỉ DDRAM của LCD. Hàm này giúp hiển thị nội dung tại vị trí đại diện cần thiết.

4 Kiểm thử

Bên trong hàm main được thiết kế như sau:

```
/* USER CODE BEGIN 2 */
timer_init();
lcd_init();
sche_init();

sche_add_task(Blinky_led, 0, 50);|
sche_add_task(TrafficLight_run, 0, 100);
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */

while (1)
{

    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    FMS_mode_run();
    sche_dispatcher();
    displayLCD();

    /////      lcd_goto_XY(1, 2);
    /////      lcd_send_string("                ");
}
```

Figure 8: Nội dung hàm main